

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи № 7
з курсу “Операційні системи реального часу”
на тему: “Обробка даних із сенсорів у багатозадачній системі”

Виконав:
Студент. гр. ФеП-41 Фіняк Олег
Перевірив:
Павлик М.Р.

Львів – 2025

Мета роботи. Навчитися організовувати взаємодію між задачами у системі реального часу, що одночасно працюють із кількома сенсорами.

```
#include <stdio.h>
#include "pico/stdlib.h"
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include "semphr.h"

#include "DEV_Config.h"
#include "OLED_1in5.h"
#include "GUI_Paint.h"
#include "SHTC3.h"
#include "SGP40.h"
#include "QMI8658.h"

// -----
// Глобальні змінні
// -----
SemaphoreHandle_t i2cMutex;

QueueHandle_t qTempHum;    // SHTC3
QueueHandle_t qVOC;        // SGP40
QueueHandle_t qIMU;        // QMI8658
QueueHandle_t qButton;     // кнопка

bool display_on = true;

// Структури для передачі сенсорних даних
typedef struct {
    float temp;
    float hum;
} TH_t;

typedef struct {
    uint32_t voc;
} VOC_t;

typedef struct {
    float acc[3];
    float gyro[3];
} IMU_t;

// -----
// ЗАДАЧА КНОПКИ
// -----
void vButtonTask(void *params)
{
    const uint PIN_BTN = 14;
    gpio_init(PIN_BTN);
    gpio_set_dir(PIN_BTN, GPIO_IN);
    gpio_pull_up(PIN_BTN);

    bool last = true;
    bool stable = true;
    int debounce = 0;

    while (1) {
```

```

    bool read = gpio_get(PIN_BTN);

    if (read != last) {
        debounce = 3;    // 3×50mc ≈150mc debounce
        last = read;
    } else if (debounce > 0) {
        debounce--;
        if (debounce == 0) {
            stable = read;
            bool pressed = (stable == false);
            xQueueSend(qButton, &pressed, 0);
        }
    }

    vTaskDelay(pdMS_TO_TICKS(50));
}

// -----
// ЗАДАЧА ТЕМПЕРАТУРА/ВОЛОГИСТЬ (SHTC3)
// -----
void vTempHumTask(void *params)
{
    SHTC3_Init();
    TH_t data;
    bool crc;
    int16_t id;

    while (1) {
        xSemaphoreTake(i2cMutex, portMAX_DELAY);
        crc = SHTC3_Measurement(&data.temp, &data.hum);
        id = SHTC3_Read_Id();
        xSemaphoreGive(i2cMutex);

        if (crc && id != -1) {
            xQueueSend(qTempHum, &data, 0);
        }

        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

// -----
// ЗАДАЧА VOC (SGP40)
// -----
void vVocTask(void *params)
{
    SGP40_init();
    VOC_t data;

    while (1) {
        xSemaphoreTake(i2cMutex, portMAX_DELAY);
        data.voc = SGP40_MeasureVOC(25, 50);
        xSemaphoreGive(i2cMutex);

        xQueueSend(qVOC, &data, 0);
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

```

```

// -----
// ЗАДАЧА IMU (QMI8658)
// -----
void vImuTask(void *params)
{
    IMU_t data;
    unsigned int tim;
    QMI8658_init();

    while (1) {
        xSemaphoreTake(i2cMutex, portMAX_DELAY);
        QMI8658_read_xyz(data.acc, data.gyro, &tim);
        xSemaphoreGive(i2cMutex);

        xQueueSend(qIMU, &data, 0);
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

// -----
// ЗАДАЧА OLED – ВИВЕДЕННЯ
// -----
void vOledTask(void *params)
{
    uint8_t *BlackImage;
    uint16_t size = ((OLED_1in5_WIDTH/2) * OLED_1in5_HEIGHT);

    BlackImage = malloc(size);
    Paint_NewImage(BlackImage, OLED_1in5_WIDTH, OLED_1in5_HEIGHT, 0, WHITE);
    Paint_SetScale(16);
    Paint_SelectImage(BlackImage);

    TH_t th = {0};
    VOC_t voc = {0};
    IMU_t imu = {0};

    bool btn;

    while (1) {

        // Перемикання екрану
        if (xQueueReceive(qButton, &btn, 0)) {
            if (btn) display_on = !display_on;
        }

        if (!display_on) {
            OLED_1in5_Clear(0);
            vTaskDelay(pdMS_TO_TICKS(200));
            continue;
        }

        xQueueReceive(qTempHum, &th, 0);
        xQueueReceive(qVOC, &voc, 0);
        xQueueReceive(qIMU, &imu, 0);

        Paint_Clear(BLACK);

        char s[32];
    }
}

```

```

    Paint_DrawString_EN(5, 5, "SENSORS", &Font12, WHITE, BLACK);

    sprintf(s, "T: %.2f", th.temp);
    Paint_DrawString_EN(5, 25, s, &Font12, WHITE, BLACK);

    sprintf(s, "H: %.2f", th.hum);
    Paint_DrawString_EN(5, 40, s, &Font12, WHITE, BLACK);

    sprintf(s, "VOC: %u", voc.voc);
    Paint_DrawString_EN(5, 55, s, &Font12, WHITE, BLACK);

    sprintf(s, "AX: %.2f", imu.acc[0]);
    Paint_DrawString_EN(5, 75, s, &Font12, WHITE, BLACK);

    sprintf(s, "GY: %.2f", imu.gyro[1]);
    Paint_DrawString_EN(5, 90, s, &Font12, WHITE, BLACK);

    OLED_1in5_Display(BlackImage);
    vTaskDelay(pdMS_TO_TICKS(250));
}
}

// -----
// MAIN
// -----
int main()
{
    stdio_init_all();
    DEV_Module_Init();
    OLED_1in5_Init();
    OLED_1in5_Clear(0);

    i2cMutex = xSemaphoreCreateMutex();

    qTempHum = xQueueCreate(5, sizeof(TH_t));
    qVOC = xQueueCreate(5, sizeof(VOC_t));
    qIMU = xQueueCreate(5, sizeof(IMU_t));
    qButton = xQueueCreate(5, sizeof(bool));

    xTaskCreate(vButtonTask, "btn", 512, NULL, 2, NULL);
    xTaskCreate(vTempHumTask, "th", 512, NULL, 2, NULL);
    xTaskCreate(vVocTask, "voc", 512, NULL, 2, NULL);
    xTaskCreate(vImuTask, "imu", 512, NULL, 2, NULL);
    xTaskCreate(vOledTask, "disp", 1024, NULL, 1, NULL);

    vTaskStartScheduler();
    while (1);
}

```

Висновок: У ході роботи створено багатозадачну систему FreeRTOS, у якій кілька сенсорних задач паралельно зчитують дані, використовуючи м'ютекс для коректного доступу до спільної I²C шини. Передавання інформації між задачами реалізовано через черги, що забезпечує повністю безпечний обмін даними. Задача дисплея агрегує результати та оновлює OLED з визначеною періодичністю. Додано керування живленням дисплея кнопкою, що дозволяє перемикає стан без переривання роботи сенсорів.